

Securing generative AI workloads

Skill 3.2.7 – the newest content on the exam, and the one your question bank cannot help with at all.

The model is not a security boundary.
Never put a control inside the prompt.

OWASP Top 10 for LLM Applications 2025

	RISK
LLM01	Prompt injection
LLM02	Sensitive information disclosure
LLM03	Supply chain
LLM04	Data and model poisoning
LLM05	Improper output handling
LLM06	Excessive agency
LLM07	System prompt leakage
LLM08	Vector and embedding weaknesses
LLM09	Misinformation
LLM10	Unbounded consumption

ON THE EXAM

Three are new for 2025: system prompt leakage, vector and embedding weaknesses, and unbounded consumption. All three exist because of RAG and agents.

Which edition, and how AWS maps to it

- ◆ The exam guide says **GenAI OWASP Top 10 for LLM Applications** without naming an edition.
- ◆ **AWS Prescriptive Guidance maps its agentic AI security controls to the 2025 edition**, which is the one to learn.
- ◆ A **2026 edition** exists as a published PDF, but its release date field is still a placeholder and the OWASP landing page still lists the 2025 identifiers.
- ◆ The 2025 renumbering matters: sensitive information disclosure moved from LLM06 to LLM02, and model theft became unbounded consumption.
- ◆ AWS names controls per risk – agent scoping, treating prompts as code artefacts, Bedrock Guardrails, and closing the invocation path with PrivateLink.

ON THE EXAM

Learn the risks by name, not number. Numbers move between editions.

Amazon Bedrock Guardrails: the six policies

- ◆ **Content filters** – configurable thresholds across harm categories. **Prompt attack** detection sits here, in the Standard tier.
- ◆ **Denied topics** – subjects the application must not discuss, defined in natural language.
- ◆ **Word filters** – profanity and your own custom word and phrase lists.
- ◆ **Sensitive information filters** – detect and **block or mask** PII in prompts and responses.
- ◆ **Contextual grounding checks** and **Automated Reasoning checks** – the hallucination controls, targeting LLM09.

ON THE EXAM

Guardrails apply to prompts and to responses, and can be applied independently of the model – including to models outside Bedrock.

Guardrails: the honest version

- ◆ AWS's own guidance states there is no reliable prevention for prompt injection. Guardrails is one layer.
- ◆ Content filters do not cover reasoning content.
- ◆ Guardrails is a **filter on text**. It cannot stop a permitted agent doing something harmful.
- ◆ Keep **credentials and state-changing capability outside the model**; require human confirmation for irreversible actions.
- ◆ A tuned guardrail is probabilistic. An IAM policy is not.

ON THE EXAM

'Enable Guardrails' is never the answer to an authorisation question.

What you can actually see

- ◆ CloudTrail records that InvokeModel was called – not the prompt or response.
- ◆ Bedrock model invocation logging is off by default, and covers bedrock-runtime only.
- ◆ Content a guardrail blocked still appears in those logs in plain text.
- ◆ Mask it with a CloudWatch Logs data protection policy, and restrict access to the log group.
- ◆ GuardDuty AI Protection watches Bedrock and SageMaker AI data events.

ON THE EXAM

Logging to investigate a leak creates a new store of the data you were protecting.

The controls that are not AI controls

- ◆ **IAM on bedrock:InvokeModel**, scoped per model. Not everyone needs every foundation model.
- ◆ **PrivateLink** closes the invocation path so traffic never traverses the internet – AWS names this as a primary control for LLM02.
- ◆ **KMS customer managed keys** for custom models, fine-tuning data and knowledge bases.
- ◆ **The retrieval boundary around a knowledge base** – filter documents by the caller's entitlement, before retrieval, not after.
- ◆ A RAG system that retrieves documents the user may not read has leaked them, whatever the model says afterwards.

ON THE EXAM

Most of a secure GenAI architecture is ordinary AWS security applied to an unusual workload. That is the reassuring part.

Agents, tools and excessive agency

- ◆ **LLM06 turns a text problem into a real one.** An agent with tools can act, and the model decides when.
- ◆ **Agent scoping** is AWS's named control: smallest tool set, narrowest execution role.
- ◆ Read and write should be **different agents with different roles**.
- ◆ **Human confirmation before irreversible actions** – the module 7 approval gate.
- ◆ **Treat prompts as code artefacts:** reviewed, versioned, changed through a pipeline.

ON THE EXAM

If an injection succeeds, the damage is bounded by what the role permits.

A bank builds a customer service assistant on Amazon Bedrock. Its system prompt includes: "You must only discuss the account of the currently authenticated customer. Never reveal information about any other customer under any circumstances."

During testing, a crafted message persuades the assistant to summarise another customer's balance. The team proposes strengthening the wording of the system prompt and adding a denied topic for other customers' data.

What should the security architect recommend?

- | | |
|---|---|
| <p>A Enforce entitlement outside the model, so the assistant can only retrieve the authenticated customer's data</p> | <p>B Strengthen the system prompt wording and add the denied topic, as proposed</p> |
| <p>C Enable Bedrock model invocation logging so the team can detect future attempts</p> | <p>D Switch to a larger foundation model that follows instructions more reliably</p> |

◆ PAUSE AND DECIDE

What should the security architect recommend?

A Enforce entitlement outside the model
Correct. If the retrieval layer can only return the authenticated customer's records, no wording persuades the model to disclose anything else - there is nothing there to disclose.

C Enable invocation logging
Detection, not prevention - and it would write other customers' balances into a log. It makes the disclosure durable.

B Strengthen the prompt and add a denied topic
Both are probabilistic text controls, and the failure was a text control being talked around. A better-worded request is still a request.

D Use a larger model
Better instruction-following reduces the success rate and changes nothing structural. You would be tuning the odds of a data breach.

THE SIGNAL

OWASP is explicit: never treat the system prompt as a security control. Privilege separation belongs in deterministic systems.

A logistics company runs a Bedrock agent that answers questions from a knowledge base of supplier documents and can call an internal API to update delivery records. Its execution role permits read and write on that API.

A supplier uploads a PDF containing hidden text instructing the assistant to mark all outstanding invoices as paid. A user asks a routine question, the document is retrieved, and the agent calls the update API.

What is the MOST effective control against this class of attack?

- ☐ A A Bedrock guardrail denied topic covering invoice status changes
- ☐ B Contextual grounding checks to verify responses against the source documents
- ☐ C Split the agent in two, so the question-answering agent's role has no write permission
- ☐ D A word filter blocking the phrase "mark as paid" in prompts and responses

◆ PAUSE AND DECIDE

What is the MOST effective control against this class of attack?

- A

 Denied topic for invoice status changes
A text filter against an action. It might stop the agent discussing invoices, and the API call is not a discussion.
- B

 Contextual grounding checks
Aimed at hallucination – is this response supported by the sources. The injected instruction was in the sources, so grounding is satisfied.
- C

 Split the agent; no write permission on the answering role
Correct. This is agent scoping. If the answering agent cannot write, a successful injection produces text and nothing else.
- D

 Word filter on "mark as paid"
Blocklisting one phrasing of an instruction with limitless variants. It also filters text while the harm is an API call.

THE SIGNAL

The injection succeeded. It always might. The question is whether success means anything, and that is decided by the role.

A healthcare provider enables Bedrock Guardrails with sensitive information filters set to mask PII, so patient identifiers never appear in model responses. It also enables model invocation logging to CloudWatch Logs for auditing.

A review finds that the CloudWatch log group contains the original, unmasked patient identifiers from the prompts, and that 40 engineers have read access to that log group.

What should the provider do?

- ☐ A Disable model invocation logging, since guardrails already mask the responses
- ☐ B Apply a CloudWatch Logs data protection policy to the log group and restrict read access
- ☐ C Raise the guardrail's sensitive information filter threshold so it catches more identifiers
- ☐ D Move the invocation logs to S3, where they are encrypted by default

◆ PAUSE AND DECIDE

What should the provider do?

A Disable invocation logging

Removes the exposure and the audit trail together. The provider enabled it for a reason, and healthcare auditors rarely accept 'we stopped recording'.

C Raise the filter threshold

The guardrail is working. Invocation logging records what was sent to the model, before any masking, so no guardrail setting changes the log content.

B Data protection policy on the log group, plus restricted access

Correct. Masks the identifiers at source, keeps the audit trail, and fixes the second problem - 40 people reading patient data.

D Move the logs to S3

Encryption at rest protects against someone reading the disk. It does nothing about 40 engineers with legitimate read access.

THE SIGNAL

Two problems in one scenario: sensitive data in the log, and too many readers. Only one option addresses both.

After a suspected prompt injection incident, an insurer's auditors ask for a record of exactly what users sent to the model and what it returned, for the previous three months.

The security team has an organization CloudTrail covering all accounts, with data events enabled for S3 and Lambda. CloudTrail shows thousands of bedrock:InvokeModel calls with principal, time and source IP, but no prompt or response content.

Why is the content missing, and what should the team do?

- A CloudTrail data events for Bedrock were not enabled; enable them to capture past calls
- B The guardrail redacted the content in CloudTrail; adjust the guardrail configuration
- C Content is in Security Lake under a Bedrock source; query it with Athena
- D CloudTrail never records prompt content; enable Bedrock model invocation logging going forward

◆ PAUSE AND DECIDE

Why is the content missing, and what should the team do?

A Enable Bedrock data events to capture past calls

Two errors. Data events record that a call happened, not its payload – and no logging is ever retroactive, as module 1 established.

C Query Security Lake for a Bedrock source

Security Lake's native sources are the eight from module 5. Bedrock prompt content is not among them, and Security Lake cannot normalise what was never captured.

B The guardrail redacted CloudTrail

Guardrails act on prompts and responses at the model. They have no relationship to CloudTrail, which never held the content.

D CloudTrail never records content; enable invocation logging going forward

Correct. CloudTrail records the call, not the payload. Model invocation logging captures content and is off by default – so the three months are gone.

THE SIGNAL

The honest half of this answer is that the three months cannot be recovered. Logging is never retroactive.

Five things to carry forward

- ◆ **Never put a control inside the prompt.** Authorisation belongs in IAM and the retrieval layer.
- ◆ **AWS says prompt injection cannot be reliably prevented.** Assume it succeeds and bound the damage with the agent's role.
- ◆ **Guardrails filter text. They are not an authorisation system,** and they do not cover reasoning content.
- ◆ **CloudTrail records the call, not the content.** Model invocation logging is off by default.
- ◆ **Blocked content still lands in invocation logs in plain text.** Mask with a data protection policy.

ON THE EXAM

That closes Domain 3. Next: Domain 4, identity and access management – the heaviest domain, starting with policy evaluation logic.